good enough for most scenarios, but it can become a serious issue for serverless workloads that are required to start almost instantaneously.

Standard Java applications also have a higher footprint cost than other stacks like Go. Cloud services are usually based on a pay-per-use model, so reducing CPU and memory footprint means cutting down costs. This section will show you how to address this issue using GraalVM and Spring Native.

16.1.1 Understanding GraalVM and native images

So far, you've used the JVM and tools provided by OpenJDK, which comes in many distributions, such as Eclipse Adoptium (previously known as AdoptOpenJDK), BellSoft Liberica JDK, and Microsoft OpenJDK. GraalVM is a newer distribution from Oracle based on OpenJDK and is "designed to accelerate the execution of applications written in Java and other JVM languages" (www.graalvm.org).

By replacing a standard OpenJDK distribution with GraalVM as the runtime environment for your Java applications, you can increase their performance and efficiency, thanks to a new optimized technology for performing JIT compilation (the GraalVM compiler). GraalVM also provides runtimes to execute code written in other languages like JavaScript, Python, and R. You can even write polyglot applications, including Python scripts in your Java code, for example.

GraalVM offers two primary operational modes. The *JVM Runtime* mode lets you run your Java applications like any other OpenJDK distribution while improving performance and efficiency thanks to the GraalVM compiler. What makes GraalVM so innovative and popular in the serverless context is the *Native Image* mode. Rather than compiling your Java code into bytecode and relying on a JVM to interpret it and convert it to machine